LLM Developer Toolkit

Master Ecosystem Map • 17 Tools • Build Order • Dependency Graph

A composable, open-source toolkit for teams building production AI systems. Each tool solves one problem sharply. Together, they form a complete LLM engineering platform.

✅ Done	🔨 Next	📋 Planned	🔮 Future	16 tools + 1 foundation

1. Strategic Overview

The LLM Developer Toolkit is a composable ecosystem of 17 focused Python tools that collectively address every layer of the LLM engineering stack — from prompt management and agent tracing to evaluation, security, and infrastructure. Each tool is independently useful and publishable to PyPI. Together, under a shared event schema, they compose into a complete platform without requiring a centralised UI or hosted service.

> **DESIGN PRINCIPLE**
> One tool, one problem, done well. No tool depends on another unless the dependency adds clear value. The ecosystem emerges from composition, not from a monolithic platform architecture. Enterprise buyers see a platform. Individual developers adopt one tool at a time.

Why a composable toolkit beats a platform

- Each tool can go viral independently — a developer who only needs llm-diff does not need to install or understand promptlock
- Three different buyer personas (individual devs, DevOps teams, enterprise ML engineers) adopt at their own pace with no forced bundling
- OSS community builds around individual tools, not a monolith — stars, issues, and contributions are focused and meaningful
- The shared schema creates enterprise stickiness without infrastructure lock-in — customers own their data and can route it anywhere
- Competitive surface is narrow per tool — each one competes in a specific niche rather than head-on with Langfuse or Arize across all features

The shared foundation: llm-schema

Every tool in the ecosystem emits and consumes the same OpenTelemetry-compatible JSON event schema. This is the single architectural decision that transforms 17 independent tools into an ecosystem. Without it, tools are isolated utilities. With it, enterprises can pipe llm-trace spans into evalkit evaluations, route promptlock audit events into their SIEM, and visualise everything in an existing Grafana or Datadog stack.

```
// Every tool emits events that conform to this envelope
{
  "schema_version": "1.0",
  "event_id":       "evt_01HX...",           // ULID
  "event_type":     "llm.span.completed",    // namespaced type
  "timestamp":      "2026-02-15T14:32:01Z",  // UTC ISO-8601
  "source":         "llm-trace@0.3.1",       // emitting tool + version
  "trace_id":       "trace_abc...",           // links related events
  "span_id":        "span_xyz...",
  "payload":        { /* tool-specific data */ },
  "tags":           { "env": "prod", "model": "gpt-4o" }
}
```

2. Master Tool List

All 17 tools, organised by category. The Schema column indicates whether the tool emits llm-schema compatible events. The Depends On column shows direct tool dependencies (not Python package dependencies).

VERSIONING & MANAGEMENT

Tool	Description	Depends On	Schema	Status
`llm-diff` *diff LLM outputs*	Compares two LLM responses word-level, semantically, or as JSON. Core evaluation primitive for the ecosystem.	`llm-schema`	✓	✅ Done
`promptlock` *prompt version control*	Git-style versioning for prompts with enterprise RBAC, immutable audit log, approval workflows, and SSO.	`llm-schema`	✓	🔨 Next
`promptblock` *prompt template engine*	Variable management, composition, and inheritance for prompt files. Renders templates with env-specific values.	`llm-schema`	✓	📋 Planned

OBSERVABILITY & TRACING

Tool	Description	Depends On	Schema	Status
`llm-trace` *agent span tracing*	Framework-agnostic OpenTelemetry-compatible span tracing for multi-step agent runs. Works with any LLM stack.	`llm-schema`	✓	🔨 Next

llm-cost *cost & latency tracking*	Per-call, per-run, per-agent USD and token accounting with budget alerts and trend dashboards.	llm-schema	✓	📋 Planned
llm-inspect *tool call inspector*	Surfaces tool call arguments, return values, and whether the model used the result correctly. DevTools for agents.	llm-schema, llm-trace	✓	📋 Planned
agentboard *local run dashboard*	Self-hosted localhost dashboard for agent run history, tool traces, and cost. No account or signup required.	llm-schema, llm-trace	—	📋 Planned

EVALUATION & TESTING

Tool	Description	Depends On	Schema	Status
evalkit *agent eval framework*	Scenario-based, framework-agnostic agent evaluation. Define test cases, run against any agent, score outputs.	llm-schema, llm-diff	✓	📋 Planned
agentping *agent health checker*	Runs a test suite against a live agent endpoint. Pass/fail per tool call. Think curl but for agents.	llm-schema	✓	📋 Planned
llm-assert *output assertions*	Assertion library for LLM outputs. Assert: contains X, matches JSON schema, sentiment > Y, latency < Z.	llm-schema	✓	🔮 Future

SECURITY & SAFETY

Tool	Description	Depends On	Schema	Status
promptguard *injection firewall*	Prompt injection detection and blocking. Input/output scanning, configurable guardrail policies, risk scoring.	llm-schema	✓	📋 Planned
llm-redact *PII redaction*	Detects and redacts PII in prompts and responses before logging. GDPR/HIPAA-ready. Plugs into llm-trace.	llm-schema	✓	🔮 Future
llm-fence *output schema enforcement*	Enforces structured output schema on LLM responses. Type-safe validation, retry on schema mismatch.	llm-schema	✓	🔮 Future

INFRASTRUCTURE & RUNTIME

Tool	Description	Depends On	Schema	Status

toolsmith *tool schema builder*	Define LLM tool schemas with Python decorators. Auto-generates OpenAI / Anthropic compatible JSON schemas.	llm-schema	—	📋 Planned
llm-retry *retry & fallback engine*	Intelligent retry with exponential backoff and cross-provider fallback. Circuit breakers included.	llm-schema	✓	🔮 Future
llm-cache *semantic caching layer*	Deduplicates near-identical prompts using embedding similarity. Cuts costs on repeated or similar queries.	llm-schema	✓	🔮 Future

SHARED FOUNDATION

Tool	Description	Depends On	Schema	Status
llm-schema *shared event schema*	OpenTelemetry-compatible JSON/OTLP event schema emitted by all tools. The contract that makes the ecosystem composable.	–	—	🔨 Next

3. Tool Profiles

Detailed profile for each tool: the precise problem it solves, its design constraints, and how it integrates with the rest of the ecosystem.

llm-schema — Shared Event Schema [BUILD FIRST]

WHY FIRST
Every other tool builds against this contract. If you change the schema after three tools are in production, you break everything. Design it correctly before writing a single line of any other tool.

- OpenTelemetry-compatible: events map cleanly to OTLP spans and logs — enterprise routing to Datadog, Grafana, Honeycomb, Splunk requires zero adapter work
- Namespaced event types: llm.prompt.saved, llm.span.completed, llm.eval.scored, llm.guard.blocked — each tool owns its namespace
- Versioned schema: schema_version field allows forward-compatible evolution without breaking consumers
- Published as a standalone PyPI package — any third-party tool can adopt it
- Includes: Python dataclasses for all event types, JSON Schema for validation, Pydantic models, serialisation helpers

llm-diff — Output Comparison [✅ DONE]

- Word-level, semantic, and JSON diff modes across any two OpenAI-compatible endpoints

- Multi-provider: OpenAI, Anthropic, Groq, Mistral, local Ollama — auto-detected from model name
- HTML report output: self-contained offline-shareable diff report
- Schema integration: each diff run emits an llm.diff.completed event with similarity score, token counts, and latency
- Ecosystem role: the evaluation primitive — evalkit and promptlock both depend on it for output comparison

promptlock — Prompt Version Control [🔨 NEXT]

- Git-style CLI: save, log, diff, rollback, tag, push, pull
- Enterprise: immutable audit log, RBAC with 7 roles, approval workflows, SSO (SAML/OIDC), SCIM provisioning
- Environment management: dev / staging / prod with configurable promotion gates
- Secrets management: variable templating, encrypted vault, HashiCorp Vault / AWS Secrets Manager integration
- llm-diff integration: auto-runs output diff on promotion request as an eval gate
- Schema: emits llm.prompt.saved, llm.prompt.promoted, llm.prompt.rolled_back events

promptblock — Prompt Template Engine [📋 PLANNED]

- Jinja2-style templating with LLM-specific extensions: model routing blocks, conditional sections by environment
- Prompt composition: build complex prompts by inheriting from and extending base templates
- Variable validation: type-check and validate template variables before rendering — catch missing variables at build time, not runtime
- Integrates with promptlock: templates stored and versioned in the registry; rendered at call time with env-specific values
- Schema: emits llm.prompt.rendered events with variable values (redacted if marked secret)

llm-trace — Agent Span Tracing [🔨 NEXT AFTER PROMPTLOCK]

> **MARKET OPPORTUNITY**
> LangSmith is the closest competitor but only works inside the LangChain ecosystem. llm-trace is framework-agnostic — works with raw OpenAI calls, AutoGen, CrewAI, LangChain, or entirely custom stacks. That is the moat.

- Instruments any Python function as an agent span with a single decorator: @trace()
- Captures: span start/end, tool calls made, tool arguments and return values, model calls within the span, branching and retry paths
- Parent-child span relationships: full trace tree of a multi-step agent run
- Exports to: local JSON files, OTLP endpoint, Datadog, Grafana Tempo, Honeycomb, Jaeger
- Async-native: works with asyncio, concurrent tool calls, parallel agent branches
- Schema: the primary producer of llm.span.* events — llm-inspect, llm-cost, and agentboard all consume these

llm-cost — Cost & Latency Tracking [📋 PLANNED]

- Per-call and per-trace USD cost calculation across all major providers — updated pricing tables maintained as a data file
- Budget alerts: configurable spend thresholds with Slack / email / webhook notifications
- Trend views: rolling 7-day and 30-day cost and latency dashboards in the terminal
- Cost attribution: tag costs by team, project, environment, or custom label
- Extracts naturally from llm-diff (already tracks per-call tokens) and llm-trace spans
- Schema: consumes llm.span.completed events and emits llm.cost.recorded events

llm-inspect — Tool Call Inspector [📋 PLANNED]

- Surfaces every tool call in an agent run: function name, arguments passed, return value, time taken
- Verifies model reasoning: did the model actually use the tool result, or ignore it? Flags cases where tool output was discarded
- Interactive replay: step through a recorded agent run tool call by tool call in the terminal
- Diff tool calls between two runs: spot when a prompt change causes different tool call patterns
- Depends on llm-trace spans — llm-inspect is the analysis layer on top of the tracing data
- Schema: consumes llm.span.* events, emits llm.inspect.report events

agentboard — Local Run Dashboard [📋 PLANNED]

- Self-hosted localhost web UI — one pip install, one command, no account or signup
- Displays: agent run history, span trees, tool call timeline, cost per run, similarity scores from llm-diff
- Filters: by model, date range, environment, tag, success/failure
- Export: any run as a shareable HTML report (same format as llm-diff reports)
- No telemetry, no cloud, no account — designed to be the zero-friction entry point for developers who resist SaaS
- Consumes llm-schema events from a local event store (SQLite by default)

evalkit — Agent Evaluation Framework [📋 PLANNED]

- Define test scenarios: a scenario is a prompt + expected tool call sequence + output assertion
- Framework-agnostic runner: evalkit does not care how your agent is built — it calls any callable
- Scoring: deterministic (schema match, tool call match) and model-graded (LLM-as-judge for qualitative outputs)
- Regression mode: run the full scenario suite and fail if any score drops below threshold vs. last baseline
- CI-native: JSON output, non-zero exit on failure, GitHub Actions action published
- Uses llm-diff for output comparison within scenario scoring

agentping — Agent Health Checker [📋 PLANNED]

- Runs a lightweight test suite against a live agent endpoint and reports pass/fail per tool call
- Think: curl for agents — one command that tells you if your agent is healthy right now
- Supports: HTTP endpoints, Python callables, WebSocket agents
- Health checks: latency threshold, response schema, required tool calls, forbidden outputs
- Integrates with monitoring: Prometheus metrics endpoint, Datadog checks, PagerDuty alerts

llm-assert — Output Assertions [🔮 FUTURE]

- Fluent assertion library: assert_that(response).contains('refund').matches_schema(schema).sentiment_above(0.5)
- Plugs into pytest: use llm_assert fixtures in existing test suites with no runner changes
- Soft assertions: collect all failures in a run rather than stopping at the first — surfaces the full picture
- Custom matchers: register domain-specific assertions (e.g., is_valid_sql(), cites_only_approved_sources())

promptguard — Prompt Injection Firewall [📋 PLANNED]

- Detects prompt injection patterns in user input before it reaches the model
- Output scanning: detects jailbreak success indicators, policy violations, and data exfiltration patterns in responses
- Configurable policies: allowlist/denylist rules, regex patterns, LLM-based detection for sophisticated attacks
- Risk scoring: every input and output gets a risk score — block, flag, or log based on configurable thresholds
- Audit integration: all block/flag events emitted as llm.guard.blocked events — feeds into promptlock audit log

llm-redact — PII Redaction [🔮 FUTURE]

- Detects PII (names, emails, phone numbers, SSNs, credit cards, addresses) in prompts and responses
- Redacts before logging: PII is replaced with [REDACTED:TYPE] tokens in llm-schema events — original never hits the event store
- Configurable entity types: choose which PII types to detect per deployment
- GDPR/HIPAA-ready: redaction runs in-process with no data leaving the environment

llm-fence — Output Schema Enforcement [🔮 FUTURE]

- Validates LLM output against a defined JSON Schema or Pydantic model
- Retry on mismatch: automatically retries with an updated prompt requesting compliant output
- Configurable retry budget: max retries, backoff, fallback value if all retries fail
- Works with any provider — does not depend on native structured output features (though it can use them)

toolsmith — Tool Schema Builder [📋 PLANNED]

- Define LLM tools with Python decorators — no manual JSON schema writing
- Auto-generates: OpenAI function calling schema, Anthropic tool use schema, LangChain tool definitions from a single decorated function
- Type inference: Python type annotations and docstrings become schema properties and descriptions automatically
- Validation: validates tool call arguments at runtime before executing the function

llm-retry — Retry & Fallback Engine [🔮 FUTURE]

- Configurable retry with exponential backoff for rate limits, timeouts, and transient errors
- Cross-provider fallback: if gpt-4o fails, fall back to claude-3-5-sonnet or a local model
- Circuit breaker: stop retrying a failing provider and route all traffic to fallback automatically
- Cost-aware routing: prefer the cheapest provider that meets latency requirements

llm-cache — Semantic Caching [🔮 FUTURE]

- Caches LLM responses and deduplicates semantically similar queries — a query 95% similar to a cached one returns the cached response
- Configurable similarity threshold: tune the aggressiveness of deduplication per use case
- Storage backends: in-memory (dev), Redis (production), SQLite (local)
- Cache invalidation: TTL-based and manual; promptlock prompt changes can trigger cache invalidation automatically

4. Build Order & Roadmap

Build order is determined by three factors: schema dependency (llm-schema must come first), market opportunity (highest-value gaps built earliest), and revenue path (tools that unlock paid tiers prioritised over purely community tools).

Phase	Timeline	Tools	Milestone	Revenue Signal
Phase 0	Now	`llm-schema`	Define the shared event contract. All tools build against this. No shipping without it.	None — foundation work
Phase 1	Weeks 1–6	`promptlock`	Prompt version control live. Enterprise design partners onboarded. First paid conversations start.	Design partner ARR
Phase 2	Weeks 7–10	`promptblock`	Template engine ships. Pairs naturally with promptlock — immediate upsell for existing users.	promptlock Pro upsell
Phase 3	Weeks 11–16	`llm-trace`	Agent tracing goes live. This is the highest-value gap in the market. Target LangSmith switchers.	First enterprise pipeline deals
Phase 4	Weeks 17–20	`llm-cost` + `llm-inspect`	Cost and tool call inspection. Natural add-ons for llm-trace users. Low build cost, high value.	Usage-based billing unlocked
Phase 5	Months 6–8	`evalkit` + `agentping`	Evaluation layer complete. evalkit + llm-diff + llm-trace = full quality assurance pipeline.	Evaluation SaaS tier
Phase 6	Months 8–10	`promptguard` + `agentboard`	Security and local dashboard. promptguard = enterprise security add-on. agentboard = OSS viral loop.	Security premium tier

Phase 7	Months 10–14	`toolsmith` + `llm-assert` + `llm-redact` + `llm-fence` + `llm-retry` + `llm-cache`	Complete ecosystem. Every tool in the stack is live and composable.	Full platform ARR

CRITICAL PATH

llm-schema → promptlock → llm-trace. These three form the critical path. If llm-schema is designed poorly, every subsequent tool needs to be refactored. If promptlock ships late, the enterprise revenue story is delayed. If llm-trace is not framework-agnostic from day one, the moat disappears.

5. Inter-Tool Dependency Matrix

A ✓ indicates that the row tool depends on the column tool. The matrix covers the 10 most interconnected tools. All tools in the ecosystem depend on llm-schema — that column will be entirely filled once all tools ship.

Tool ↓ Uses →	schema	diff	promptlock	trace	cost	inspect	evalkit	agtping	promptguard	toolsmith
`llm-schema`	■	·	·	·	·	·	·	·	·	·
`llm-diff`	✓	■	·	·	·	·	·	·	·	·
`promptlock`	✓	·	■	·	·	·	·	·	·	·
`llm-trace`	✓	·	·	■	·	·	·	·	·	·
`llm-cost`	✓	·	·	·	■	·	·	·	·	·
`llm-inspect`	✓	·	·	✓	·	■	·	·	·	·
`evalkit`	✓	✓	·	·	·	·	■	·	·	·
`agentping`	✓	·	·	·	·	·	·	■	·	·
`promptguard`	✓	·	·	·	·	·	·	·	■	·
`toolsmith`	✓	·	·	·	·	·	·	·	·	■

Key observations from the dependency graph:

- llm-schema is the single most depended-upon tool — it appears in every row. This is by design.
- llm-trace is the second most depended-upon — llm-inspect, agentboard, and evalkit all build on top of it. Build it with a stable API from day one.
- llm-diff is a dependency of evalkit — the evaluation framework uses output comparison as a primitive. This is already built.
- No circular dependencies — the graph is a clean DAG. toolsmith, agentping, promptlock, and promptguard are leaves — they depend on the foundation but nothing depends on them.

6. Monetisation Map

Every tool is OSS at its core. Revenue is generated through the SaaS platform (hosted registry, team features, enterprise governance) and enterprise-only features that are not in the OSS repos.

Tool	OSS Revenue	SaaS / Pro Revenue	Enterprise Revenue
`llm-diff`	OSS / community	Batch compare SaaS tier	CI/CD integration + team report sharing
`promptlock`	OSS CLI (MIT)	Pro: 5 users, environments	Enterprise: RBAC, audit log, SSO, SCIM, BYOK
`promptblock`	OSS (MIT)	Pro: team template library	Enterprise: org-wide template governance
`llm-trace`	OSS (MIT)	Hosted trace storage + search	Enterprise: 1yr retention, SIEM export, custom exporters
`llm-cost`	OSS (MIT)	Team cost dashboards + alerts	Enterprise: org-wide cost allocation, chargebacks
`llm-inspect`	OSS (MIT)	Hosted replay + team sharing	Enterprise: audit-grade tool call logging
`evalkit`	OSS (MIT)	Hosted eval history + regression	Enterprise: custom eval suites + compliance reporting
`promptguard`	OSS core	Pro: managed policy updates	Enterprise: custom policies, SOC 2, HIPAA compliance
`agentboard`	OSS / always free	Cloud sync + team sharing	Enterprise: SSO, audit, multi-env dashboards
`toolsmith`	OSS / always free	Hosted schema registry	Enterprise: org-wide tool registry + governance

7. Ecosystem Success Metrics

Metric	6 Months	12 Months	24 Months
Total GitHub stars (all repos)	**5,000+**	**25,000+**	**100,000+**
Total PyPI downloads / month	**20,000+**	**200,000+**	**1M+**
Tools live on PyPI	**4**	**10**	**17**
Enterprise design partners	**5**	**15**	**50+**
Enterprise ARR	**$250K**	**$1.5M**	**$5M+**
OSS contributors	**20+**	**100+**	**500+**
llm-schema adopters (3rd party tools)	**0**	**5+**	**25+**
Companies routing to Datadog via schema	**3**	**20+**	**100+**

THE ONE RULE

Ship llm-schema before anything else. Every other decision is reversible. A poorly designed shared schema is not. Spend a full week on it. Get three people with different use cases to review it. Then build everything else on top of it with confidence.